

Implementation Plan: SANKET — Foundation & Test Version (Prompt 1 of 12)

Building **SANKET** (AI Exam Invigilation Assistant) per the handoff spec, for Problem Statement #1 (*AI-Based Human Behaviour Monitoring Using Pose Estimation and Gesture Analysis*). Target hardware: Apple Silicon M3, PyTorch MPS backend. No CUDA, no profile ladder.

Discipline Gate — read before touching code

[!IMPORTANT] **This plan builds Prompt 1 only.** Twelve prompts exist total (foundation → pose/tracking → seat anchoring → calibration → features → rules/scoring → objects → store/reports → backend → dashboard → clips/eval → integration freeze). The handoff is explicit: *"Go step-by-step, never build two prompts before testing one, and do not over-extend."*

Gate conditions, in order:

Paste the handoff's Part A context pack into the agent session first — it carries invariants (time-base rule, named-keypoint rule, config-only thresholds, "cheating" is never used, etc.) that every later prompt assumes and this plan does not repeat.

Build everything below. Nothing from Prompt 2 (pose, tracking, skeletons) belongs in this pass — no early imports of `ultralytics`, no stub `pose.py`. If it's tempting to add "just the skeleton", that's scope creep into a prompt you haven't started.

Run the full Verification Plan (all 5 checks, not a subset). All 5 must pass before anything else happens.

`git push` once verification passes — the handoff treats this as mandatory after every completed prompt, not optional housekeeping, because it's what makes two-machine parallel work on later prompts safe.

Only then does Prompt 2 (Pose Estimation & Tracking) start, in a new pass.

Architecture & System Invariants

Time Base Invariant: All timestamps and durations derive strictly from the source ($t = \text{frame_index} / \text{fps}$), never wall-clock time for recorded media. This is the single most load-bearing rule in the whole codebase — every later prompt's scoring decay depends on it being right now.

Data Contract: Standard schema for Events, Seat States, Sessions, and API endpoints, frozen and saved in `DATA_CONTRACT.md` at repo root. This doesn't get used until Prompt 8+ but it's written now, at hour zero, so backend and frontend work (Prompts 9–10) can build against it independently later without waiting on each other.

Hardware & Environment: MPS / CPU device resolution with graceful fallback. `half=False` — fp16 is unreliable on MPS. No CUDA detection code anywhere; this project targets Apple Silicon only.

Clean Code & Separation:

`config.yaml` manages all thresholds and settings. Nothing hardcoded outside it, even placeholder values.

`sanket/render.py` is the only module responsible for drawing on frames.

`sanket/source.py` abstracts video files, RTSP/HTTP streams, and webcams uniformly — one code path for all three, so a live-CCTV demo and a file replay are indistinguishable to everything downstream.

Ethical & Terminology Standard: Strictly use the vocabulary *alert*, *observed behaviour*, *review*. The word "cheating" is never used, including in code comments, log messages, or this plan.

User Review Required

[!IMPORTANT]

This plan covers **Prompt 1 (Foundation and Video Input)** only, of 12.

No pose estimation or tracking is loaded here; the goal is rock-solid video ingestion, timestamping, CLI harness, config parsing, and HUD rendering. It deliberately looks unimpressive — that's the point of doing the boring, breakable parts first.

Do not start Prompt 2 (Pose Estimation & Tracking) until the Verification Plan below passes in full and the result has been reviewed.

Proposed Changes (Prompt 1: Foundation & Video Input)

Root Configuration & Documentation

[NEW] **DATA_CONTRACT.md**

Define the frozen schemas: Event, Seat state, Session state, and API endpoints as specified in Part B of the handoff. Copy it verbatim — this is a contract other prompts build against, not a draft to improve on here.

[NEW] **config.yaml**

Define the complete configuration structure:

`source`: `fps_override`, `resize_width` (1280), `reconnect_max_seconds` (30), `rtsp_buffer_size` (1)

`model`: `pose_weights`, `imgsz` (640), `conf` (0.25), `keypoint_min_conf` (0.5), `device` (auto), `half` (false), `frame_skip` (1)

`identity`, `calibration`, `rules`, `objects`, `scoring`, `output` placeholders with default parameters —

empty/minimal now, populated by the prompt that actually needs them (Prompts 3-8). Don't pre-fill values for rules that don't exist yet; a placeholder with an invented threshold is indistinguishable from a real one six prompts later.

[NEW] **requirements.txt**

Pin core dependencies: `opencv-python`, `numpy`, `pyyaml`, `ultralytics`, `torch`, `fastapi`, `uvicorn`, `pydantic`.

`ultralytics`/`torch`/`fastapi`/`uvicorn`/`pydantic` aren't used until later prompts — pinned now so the environment is settled once, not re-resolved under time pressure mid-hackathon.

Core Package (`sanket/`)

[NEW] **sanket/__init__.py**

Package initialization and version definition.

[NEW] **sanket/log.py**

Structured logging utility.

[NEW] **sanket/config.py**

Config loader and validator with dictionary attribute access and defaults fallback.

[NEW] **sanket/device.py**

`resolve_device(pref)` returning `"mps"` or `"cpu"`, via `torch.backends.mps.is_available()`.

Sets `PYTORCH_ENABLE_MPS_FALLBACK=1` on import so unsupported ops fall back to CPU instead of crashing. Prints the chosen device and *why* at startup — this becomes a quoted figure in Prompt 12's `FIGURES.md`, so make the log line copy-pasteable.

[NEW] `sanket/source.py`

`Frame` dataclass (`index: int`, `t: float`, `image: np.ndarray`).

`FrameSource` context manager handling file paths, RTSP/HTTP URLs, webcam index integers, via

`open_source(spec: str)`.

Calculates $t = \text{index} / \text{fps}$ for recorded video; wall-clock elapsed since start for live streams — document this asymmetry in a comment where it's easy to miss later.

Honours `source.fps_override` (some CCTV files report the wrong fps).

Aspect-ratio-preserving downscale to `resize_width` as a **ceiling cap** (never upscaling), exposing `.scale` so coordinates map back to original resolution.

For RTSP specifically: `CAP_PROP_BUFFERSIZE = source.rtsp_buffer_size` so the newest frame is always processed rather than a growing backlog; reconnect with **exponential backoff** up to `reconnect_max_seconds`; every dropped interval is recorded into `.gaps` as `(t_start, t_end)` so later reports can show which minutes went unmonitored. This is the concrete answer to the "Network Failure" risk row in the problem statement — don't skip it as a nice-to-have.

Context manager always releases the capture, including on exception.

[NEW] `sanket/render.py`

HUD drawing function: `draw_hud(frame, info: dict)` — compact, jitter-free, fixed-width top-left overlay (frame index, timestamp `MM:SS.mmm`, fps, source name). Fixed-width matters more than it looks: the dashboard's "layout stability" requirement in Prompt 10 starts here.

Main Entrypoint

[NEW] `main.py`

CLI entrypoint: `python main.py --source <spec> [--show] [--max-frames N] [--config path]`.

Iterates frames, renders HUD, optionally displays video preview with `q` to exit.

On completion, prints: frames processed, source duration, wall time, mean processing fps, resolved device, detected source fps, and any recorded gaps.

Verification Plan

These are the handoff's actual Prompt 1 acceptance criteria — run all 5, not a convenient subset:

File source, display: `--source <a real file> --show` plays with HUD visible and correct.

Webcam source: `--source 0 --show` works against the built-in camera.

Frame cap: `--max-frames 100` stops at exactly 100 frames, no off-by-one.

Clean failure: a missing or corrupt path produces a one-line error message, never a raw traceback.

Device/fps reporting: startup output prints the resolved device (`mps`/`cpu`) and the source's detected fps. Additional check specific to the time-base invariant (not in the handoff's short list, but load-bearing enough to verify explicitly here):

Time base: run on a 10-minute clip and confirm the HUD timestamp reads `10:00` at the final frame — this is the cheapest possible proof that $t = \text{index} / \text{fps}$ is implemented correctly, before six later prompts build

decay and duration logic on top of it.

No pose estimation, no tracking, no AI in this prompt. The only goal is frames flowing reliably, with correct timestamps, through a config-driven pipeline. If all 6 checks above pass — and only then — proceed to Prompt 2.